Private context and shared resources

A thread has its own PC, registers, stack, scheduling state, and identity. Threads usually share process code, globals, heap, address space, and open resources. One bad pointer can corrupt another thread's data. Separate stacks describe logical ownership, not hardware isolation within one address space. Another thread can access a passed stack pointer while its object remains alive.

Worker data	W1	W2	Coordination
Register/local index	Own index 0	Own index 0	Independent context
Scratch	B1 holds 110	B2 holds 120	Disjoint ownership
Shared queue	Takes job 11	Takes job 12	One protocol prevents duplicate removal
Global total	Adds 110	Adds 120	Atomic/mutex, or one reducer after joins

Responsiveness can come from a UI thread continuing while workers await I/O. Sharing can reduce some process-creation costs. Scalability needs cores and separable work. These are conditional benefits, not fixed speed factors. More threads do not remove locks, bandwidth limits, or serial dependencies.

Globals, TLS, and lifetime

Unsynchronized increments of one non-atomic C/C++ counter are a data race and undefined behavior. Read 0/read 0/write 1/write 1 is an abstract lost-update witness, not the language's unique output.

With per-thread TLS counters, each independently goes 0→1. Each worker copies its value into a distinct result slot that stays alive before it exits. Main joins both workers successfully, then reads the slots and sums 2; it does not access a destroyed TLS object. TLS persists across functions for the thread's lifetime, unlike one function's automatic local. A pool worker does not reset TLS per job automatically; initialize and clean it at job boundaries. A mutex cannot make a pointer to an ended thread's stack-local result valid.

Concurrency and decomposition

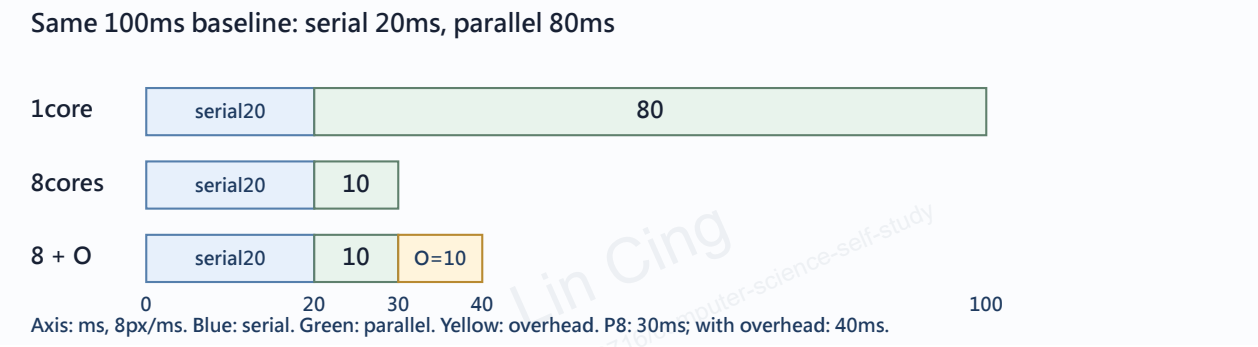
Concurrency permits overlapping or interleaved work, even on one CPU. Parallelism requires simultaneous hardware execution resources. SMT contexts share core resources; logical CPUs are not necessarily independent full cores.

Data parallelism applies one operation to partitions. Task parallelism overlaps functions or stages. Decomposition must address partitioning, balance, dependencies, sharing, and testing. A 64 MiB array of 32-bit integers contains 16,777,216 elements. Eight partitions each contain 8 MiB or 2,097,152 elements. Each worker uses a sufficiently wide 64-bit accumulator. Seven additions combine eight partials. Every element must still be read.

Per-element global atomic addition instead performs 16,777,216 shared RMWs. Publishing eight partials changes operation counts, not a measured speedup. Bandwidth, caches, and NUMA placement still matter.

Amdahl uses baseline time

For the same fixed problem, T1=100 ms contains 20 ms serial and 80 ms ideally divisible among P cores, without overhead. With s=0.2, $T_P = 100[s + (1 - s)/P]$, and speedup is $1/[s + (1 - s)/P]$. At P=8, T8=30 ms, speedup 10/3, efficiency 5/12. As P grows, speedup$\leq$5.

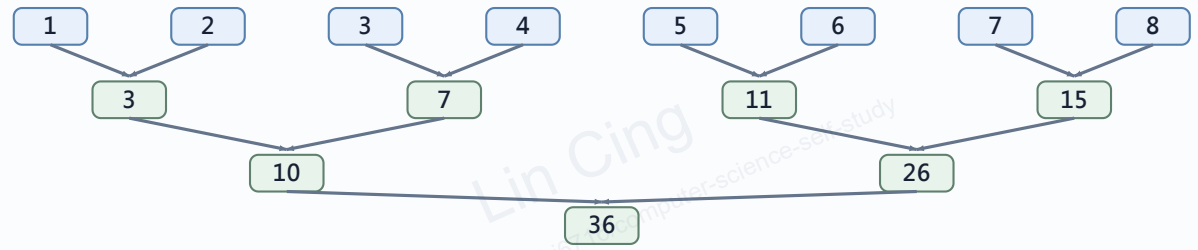


Adding 10 ms launch/synchronization overhead makes T8=40 ms and speedup 2.5. Eighty percent of operations need not mean 80% of baseline time. Eighty 1-cycle and twenty 10-cycle operations cost 280 cycles. Accelerating only the first group eightfold gives 10+200=210 cycles, speedup 4/3. Retain one denominator.

Work, span, and reduction

Eight leaves read values 1…8, and seven binary merges add them. Each leaf/merge costs one unit. Work=15. The longest path leaf→pair→quartet→root has span 4. Every P-processor schedule has $T_P \geq \max(15/P, 4)$, so speedup≤15/4. Work/span is average parallelism, not a theorem that processors beyond 3.75 cannot help.

Sum 1…8: 8 leaves +7 merges; work 15/span 4, unit node cost



Arrows: data dependencies. Each merge waits for both children. Actual partial sums; TP≥max(15/P, 4).

Pair sums are 3,7,11,15; quartets 10,26; root 36. Exactly seven merges occur. Root must await both quartets. Unequal partitions impose a lower bound equal to the largest partition time, not the average.

False sharing differs from a data race

With 64 B lines, 4 B integers, and a 64 B-aligned base, W1 writes a[0] and W2 a[1]. Distinct objects may avoid a language data race while repeatedly transferring one coherence line's ownership. Separately aligned regions at least 64 B apart can reduce interference; measurement is required to claim improvement.

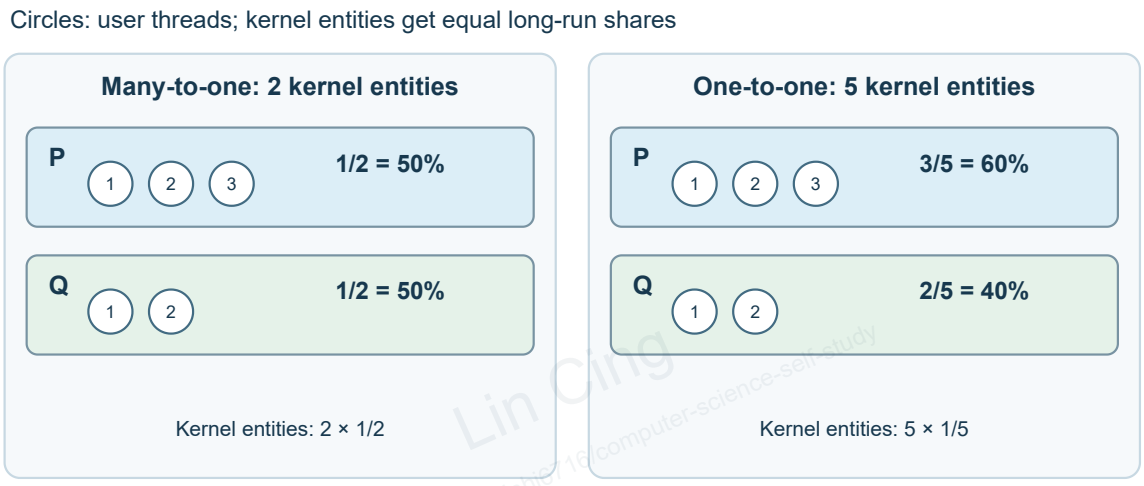
A mutex can add serialization. Replacing a shared counter with TLS changes abstract data and still needs reduction. Correctness, line-sharing cost, and result combination are separate obligations.

User-to-kernel mappings

Model	Mapping	Real blocking syscall	Parallelism
Many-to-one	Many user threads, one carrier	Without async substitution, blocks all execution there	One carrier
One-to-one	One kernel entity per thread	Other entities remain schedulable	Ready entities and cores
Many-to-many	Runtime multiplexes carriers	Depends on carriers and compensation	Runnable carriers and cores
Two-level	M:N with selected bound threads	Bound/unbound rules differ	Binding restricts runtime choices

A Pthreads API does not choose one universal mapping. Many user threads do not imply equal kernel-object counts. Nonblocking I/O and an event loop can leave many tasks logically waiting without blocking carriers.

Scheduling entities determine the modeled CPU shares



Assume one CPU, continuously runnable threads, equal priority, and long-run equal CPU shares among runnable kernel scheduling entities. Process P has three user threads; Q has two.

In this many-to-one example, each process has one carrier. The two kernel entities each receive 1/2, so P and Q get 1/2 each. With one-to-one mapping, five kernel entities each receive 1/5: P receives 3/5 in total and Q receives 2/5.

A different fairness rule changes the answer

Keep the single CPU and five continuously runnable threads. Now divide CPU equally between process groups P and Q first, then equally among threads in each group:

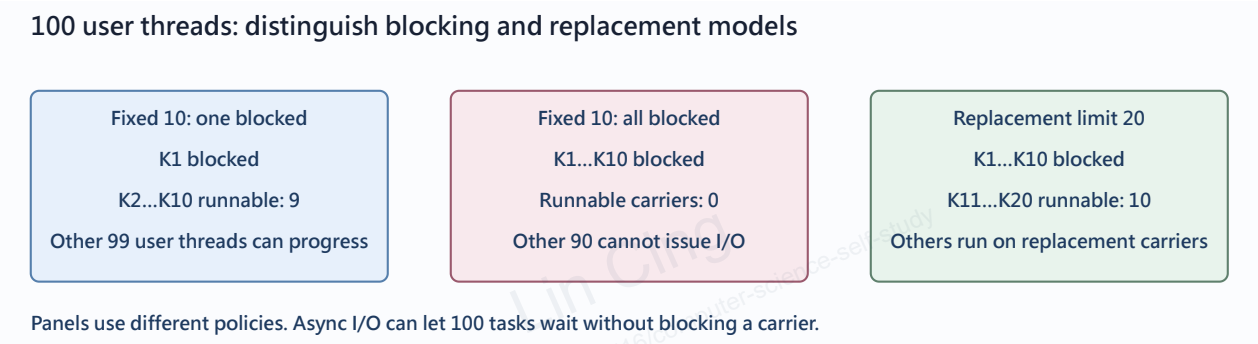
Policy	Total P	Each P thread	Total Q	Each Q thread
Five kernel entities get 1/5 each	3/5	1/5	2/5	1/5
Two process groups get 1/2 each	1/2	1/6	1/2	1/4

Over a modeled 60 ms, group fairness assigns 30 ms to each process; each P thread gets 10 ms and each Q thread gets 15 ms. This calculation excludes switching overhead.

Different weights, blocking, or multiple CPUs can also change the outcome. In many-to-one the runtime determines how to divide P's 1/2 among its user threads; the kernel share alone does not determine that division.

One hundred user threads, ten carriers

Fix K1…K10, no compensation or async I/O, and 100 threads each wanting blocking I/O. After U1 blocks K1, nine carriers remain. After ten block, none runs. The other 90 threads stay ready inside the runtime and cannot issue their syscalls.



A separate policy provides replacement carriers up to 20. K11…K20 can run after K1…K10 block and can issue additional blocking calls. Async I/O can let all 100 tasks wait while no carrier blocks. A ten-waiter bound is conditional, not universal M:N behavior.

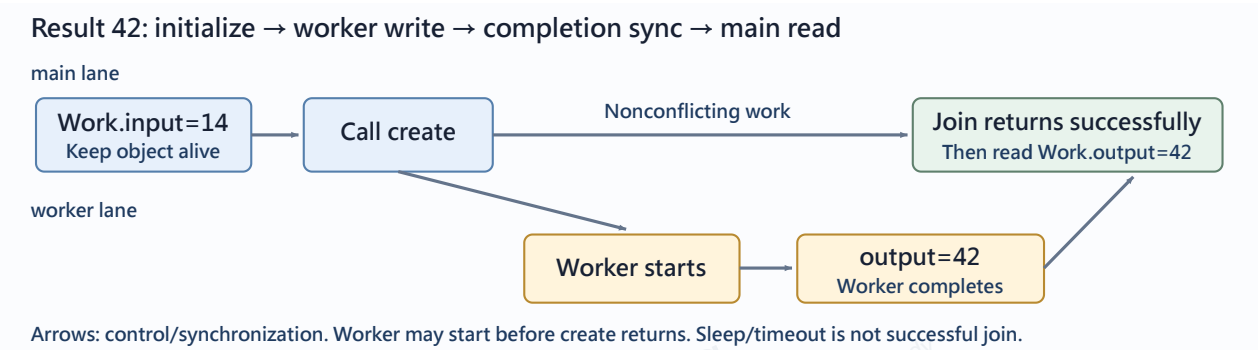
Historical scheduler activations/upcalls coordinate kernel resource notifications with runtime selection. They need rules for runtime locks, notification context, and allocation. They create no CPU capacity. Binding does not let one thread run on two carriers simultaneously.

Creation, completion, and join

Main owns Work{input14,output}; worker writes output=3×input=42. Initialize input before create, keep Work alive until successful join, and do not read output before completion synchronization. After create, the worker may already have started or finished.

Stage	Main	Worker	Result rule
Before create	Initialize 14	Absent	Valid object and arguments
After success	Nonconflicting work	May start, finish, write 42	No fixed parent/worker order
Worker complete	May await join	No longer uses Work	Publication follows library contract
Join success	Read 42, then end lifetime	Finished	No overlapping write

Creation, completion, and join (continued)



Join is not a fixed sleep. Sleeping 10 ms proves no completion. Detached threads need another lifetime/notification protocol. Detach does not authorize immediate parameter free.

Pthreads, Windows, and Java

Pthreads uses pthread_create with a routine and void* argument. Successful pthread_join waits for a joinable target and obtains its result. Handle its returned error number, not arbitrarily errno. Define join ownership; multiple joiners are not a generally safe broadcast. Never return a pointer to the worker's dead stack local.

Windows uses CreateThread and a handle. Read 42 after successful signaled WaitForSingleObject, not timeout or WAIT_FAILED. CloseHandle remains necessary and does not terminate the thread. Synchronization supplies ordering; volatile is insufficient.

Java 8 Thread.start creates execution; directly calling run is an ordinary method call on the current thread. Pre-start actions happen-before the worker. Worker actions happen-before successful join return. InterruptedException does not establish completion. API/language contracts and host mapping are separate layers.

Pools and fork-join

Eight 1-unit jobs on four FIFO workers with no overhead run J1–J4 in [0,1), J5–J8 in [1,2), makespan 2. Worker reuse avoids per-job creation, but bounded queues, backpressure, cancellation, and errors remain. Worker count and queue capacity limit different resources.

Fork-join splits independent subtasks and combines after completion. The sum tree gives 3/7/11/15→10/26→36. Too-fine granularity can make task, queue, and synchronization costs exceed useful work.

In a two-worker pool, A and B each submit A1/B1, then block waiting. Children are ready but no worker is free. This starvation deadlock can occur with an acyclic task DAG. Help-join/work stealing may execute dependencies while waiting, but arbitrary pools and Futures do not promise it.

OpenMP and GCD

OpenMP parallel-for partitions iterations. Independent c[i]=a[i]+b[i] can use distinct i; shared sum+=a[i] needs reduction/synchronization. Assume four workers actually receive [1,2], [3,4], [5,6], and [7,8]; their partial sums are 3, 7, 11, and 15. One possible merge is 10+26=36; OpenMP does not guarantee this partition or reduction order. Reduction is not arbitrary concurrent plain-sum writes. Floating-point reassociation can change final bits.

Historical GCD queues submit blocks to managed workers. A serial queue completes one at a time. Concurrent queues allow overlap; FIFO enqueue does not imply FIFO completion. A block on serial Q that synchronously submits another to Q and waits creates self-wait. Different queues do not automatically exclude shared-object access.

Runtime scheduling leaves dependencies, shared/private data, lifetime, and unsafe blocking to the application. A directive cannot erase loop-carried dependencies.

Multithread fork and signals

T2 holds M when T1 forks in a POSIX-style process. Child retains only T1, but copied M can appear owned by absent T2. A pre-exec library call needing M can stall. Before exec, use the required async-signal-safe operations. Atfork requires coordinated library/lock design and cannot universally repair unknown third-party locks.

Disposition is generally process-shared; mask is per thread. Synchronous faults target their causing thread. Process-directed signals go to an eligible thread under mask/implementation rules, not once per thread. Dedicated sigwait needs coordinated masks elsewhere. Signals and condition variables are different objects.

Successful exec replaces the image; other threads do not continue the old image. Ordinary thread creation shares it. These choices determine lock/resource inheritance.

Deferred cancellation

Successful pthread_cancel queues a request. The target may disable cancellation or not reach a point. Deferred cancellation acts at specified points. Asynchronous cancellation can interrupt broken invariants or lock ownership.

Worker owns M and buffer B and waits on a stable predicate in pthread_cond_wait. With cancellation disabled, establish resources and push unlock-M then free-B cleanup handlers. Enable deferred cancellation. Cancellation during wait first reacquires M, then runs LIFO cleanup, freeing B and unlocking M. Main successfully joins before reclaiming joinable state. Do not enable arbitrary cancellation before registration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cleanup invariants

State	M	B	Free count	Unlock count
Resources/handlers ready	Owned	Live	0	0
Cond wait	Atomically released	Live	0	0
Cancellation handled	Reacquired	Live	0	0
First cleanup	Owned	Freed	1	0
Second cleanup	Unlocked	Freed	1	1

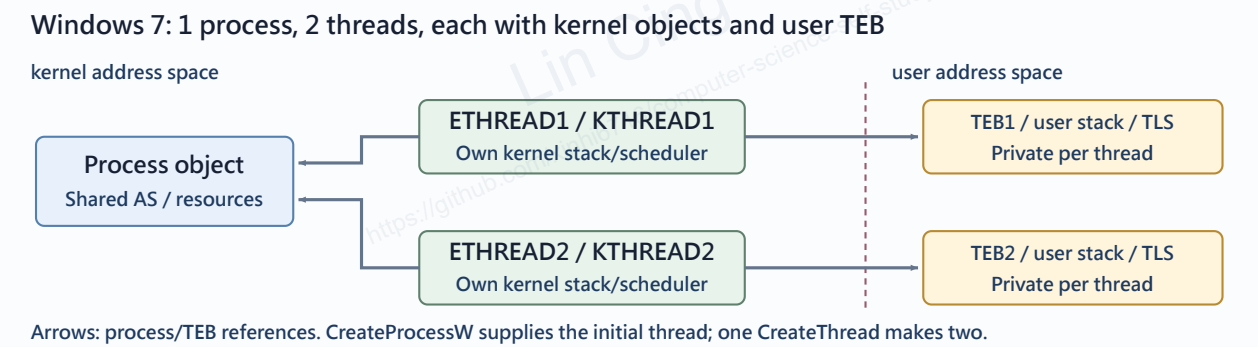
Normal execution also pops or cleans exactly once. Normal free followed by cancellation free is wrong. Releasing locks cannot repair arbitrary partial updates. Disable cancellation or avoid points while invariants are broken, or provide rollback. Include normal, error, and timeout ownership paths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Windows 7 and Linux 3.2 objects

Fix the textbook snapshot. Windows 7 processes own address spaces/resources; threads have ETHREAD, KTHREAD, and user TEB. CreateProcessW creates an initial thread; one additional CreateThread gives two one-to-one threads. Session 0 system threads are outside this ordinary interactive-process example.

Linux 3.2 task_struct represents every schedulable task. Fork's copy_process→dup_task_struct creates a new task/kernel stack. Without CLONE_VM, copy_mm→dup_mm creates independent mm/page tables. Clone flags choose sharing of fs_struct, files_struct, sighand_struct, and mm.



Windows 7 and Linux 3.2 objects (continued)

Aspect	Windows 7	Linux 3.2
Process creation	CreateProcessW	fork
Thread-like creation	CreateThread	clone with VM,FS,FILES,SIGHAND,THREAD
Execution objects	Process and thread	One task_struct per stream
Private state	Registers, stacks, TEB, TLS	Kernel stack, scheduling, registers/stack
Shared state	AS and process resources	mm, files, fs, handlers per flags
Scheduling unit	Kernel thread	Task

One process/two streams gives Windows 1 process, 2 ETHREAD, 2 KTHREAD, 2 TEB, 1 AS, 2 scheduling entities. Ordinary Linux fork gives 2 tasks, 2 mm, 2 page tables. Specified thread-like clone gives 2 tasks, 1 mm, 1 file table, 1 handler table, 2 kernel stacks.

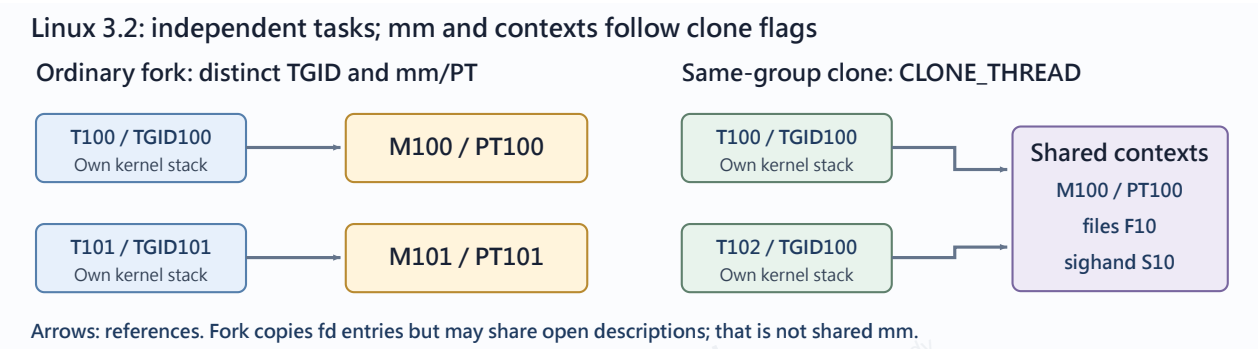
References and version counterexamples

ETHREAD references its process; KTHREAD tracks scheduling/synchronization; TEB is user space. The second thread owns independent stacks, registers, and TEB while referring to the same process/AS. Identity remains independent.

Fork creates child PID, state, stack, mm, and tables before private-page copying. COW sharing concerns physical frames, not a shared task object. Exec retains identity and changes image. Failure before destructive transition usually returns an error; some v3.2 failures after flush_old_exec terminate execution, so the old image is not unconditionally preserved.

Thread-group clone includes CLONE_THREAD, different TIDs, and one TGID. THREAD requires SIGHAND, which requires VM. VM/FS/FILES/SIGHAND with SIGCHLD may share resources without one thread group and is not a complete pthread implementation. Shared heap access still needs synchronization; for a still-shared COW page after fork, a write fault may require a private physical copy, but the kernel can sometimes reuse an exclusive frame.

References and version counterexamples (continued)



These counts describe fixed historical models, not today's kernel ABI guarantees.

Results, completion, and dependencies

Sum 1…8=36 can use one thread, four partition workers, or fork-join. Scheduling, objects, and synchronization costs differ. Read partials after joins, combine with the stated operation, and preserve lifetime.

Amdahl uses baseline-time fractions; work/span bounds a DAG; carriers bound kernel-schedulable streams. These are different bottlenecks, not proof that threads=cores is optimal or processors above average parallelism never help.

When implementation assumptions change

Changing 1:1 to M:N requires blocking/compensation analysis. Changing scratch to global requires conflict analysis. Detach replaces join's lifetime/completion proof. Multithread fork adds inherited-lock risk. Cancellation adds resource and abstract-state recovery.

For measurements, fix inputs/results, cores/carriers, memory placement, and time denominator. Retain failures and wait locations; mean throughput can hide lock contention, pool exhaustion, and cleanup tails.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 4. Linux 3.2 fork.c; pthread_join(3); pthread_cond_wait(3p); C++ thread storage; OpenMP reduction.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study